

Luna Samantha Arrazola Escobar
Rosario Isabel Palma Navas
Shayla Mayré De León Monterroso
Jeimy Raquel Palencia Sazo

Carnet: 1022725
Carnet: 1071625
Carnet: 1342925
Carnet: 1021625

Investigación Stacked Pull Request

¿Qué es un stacked pull request?

Un stacked pull request (o PRs apilados) es una forma de organizar el trabajo en git que se trata de dividir una función grande o compleja en varios pull requests pequeños, ordenados en secuencia y conectados entre sí como si fueran una torre o pila (*stack*).

En lugar de crear una sola rama grande con miles de líneas de código y mandarla a revisar junta, el desarrollador crea una serie de ramas donde cada una depende de la anterior:

- PR #1 (base): Contiene los cambios de la base de datos (apunta a la rama main).
- PR #2 (intermedio): Contiene la lógica del backend (apunta a la rama del PR #1).
- PR #3 (final): Contiene la pantalla o interfaz (apunta a la rama del PR #2).

De esta manera el equipo puede revisar y aprobar el proyecto por partes lógicas en lugar de procesar un solo bloque masivo de código.

El problema que resuelve

Para entender por qué se usan los stacked PRs hay que ver los dos problemas principales del método tradicional (*feature branching*):

1. El “mega PR”: Trabajar semanas en una rama y enviar un pull request enorme con más de 1,000 líneas de código. Esto provoca que a los compañeros les dé pereza revisar el código, hagan revisiones superficiales para terminar rápido o se pasen por alto errores graves.
2. El bloqueo entre desarrolladores: Para evitar el “mega PR”, un desarrollador envía solo la primera parte (el PR #1). Sin embargo, mientras espera a que sus compañeros lo revisen y aprueben, no puede continuar con el PR #2 sin enredar sus ramas, lo que genera tiempos muertos o cambios de contexto constantes.

Con stacked PRs, el desarrollador puede seguir programando el PR #2 y el PR #3 sobre sus propias ramas locales sin tener que esperar a que el PR #1 sea aprobado para continuar.

¿Cómo funciona en la práctica?

El flujo de trabajo sigue estos pasos:

1. Creación de la pila: Se crean las ramas en orden descendente. La rama 1 sale de main, la rama 2 sale de la rama 1, y la rama 3 sale de la rama 2.
2. Apertura de PRs en GitHub: Al subir los cambios a GitHub, se configura la “rama base” (*target branch*) de cada PR para que apunte a la rama anterior en lugar de a main.
3. Revisión incremental: Los revisores analizan bloques pequeños (100 a 200 líneas). Esto hace que las revisiones sean mucho más rápidas y precisas.
4. Fusión en cadena: Cuando el PR #1 se aprueba y se fusiona (*merge*) en main, GitHub actualiza automáticamente la base del PR #2 hacia main, continuando el proceso hasta cerrar la pila.

Ventajas y desventajas

Ventajas

- Revisiones más rápidas: Es más fácil y rápido revisar 150 líneas de código que 1,500.
- Menos errores: Las revisiones pequeñas permiten detectar fallos de lógica o seguridad que se perderían en un PR gigante.
- Trabajo continuo: El desarrollador no se queda congelado esperando aprobaciones para seguir avanzando.

Desventajas

- Complicación con Git (rebase en cascada): Si alguien pide un cambio en el PR #1, hay que hacer un git rebase para actualizar el PR #2 y el PR #3. Si no se domina Git, esto puede generar conflictos molestos.
- Falta de soporte nativo perfecto: Aunque GitHub deja cambiar la rama base manualmente, no tiene una interfaz especializada para ver toda la “pila” de un solo. Por eso muchas empresas usan herramientas extra como *Graphite* o *Sapling* para automatizar el proceso.

Conclusión

El enfoque de stacked pull requests es una buena práctica cuando se trabajan proyectos grandes o en equipos con flujos de integración continua exigentes. Requiere un poco más de disciplina y conocimiento en el manejo de ramas en Git pero a cambio ofrece revisiones de código mucho más fluidas, menos errores en producción y un ritmo de trabajo sin interrupciones.

Enlaces y fuentes confiables

- Documentación oficial de GitHub (cambiar la rama base de un PR):

Muestra cómo configurar un PR para que apunte a otra rama que no sea main.

<https://docs.github.com/es/pull-requests/collaborating-with-pull-requests/proposing-changes-to-your-work-with-pull-requests/changing-the-base-branch-of-a-pull-request>

- Guía de Graphite sobre Stacked PRs:

Explicación clara del concepto, beneficios y cómo manejar la sincronización de ramas.

<https://graphite.dev/guides/stacked-pull-requests>

- Documentación de Sapling (Herramienta de Meta / Facebook):

Explica el concepto de *stacking* desde el punto de vista de ingeniería de software a gran escala.

<https://sapling-scm.com/docs/introduction/stacking/>